
Autor

Daniel Guerrero

Especialista Técnico

Repositorio Git <https://github.com/guarrior-labs/cp2-devops-azure-unir>

Tabla de contenido

- ?? Introducción
- ?? Objetivos
- ?? Repositorio Git
- ?? Arquitectura General
- ?? Diagramas de la solución
- ?? Recursos desplegados en Azure
- ?? Descripción del proceso de despliegue
- ?? Aplicaciones desarrolladas
- ?? Automatización implementada
- ??? Hallazgos y soluciones implementadas
- ??? Resultados obtenidos
- ??? Licencia
- ??? Conclusiones
- ??? Referencias

1. Introducción

El presente documento describe el desarrollo del Caso Práctico 2 del Diplomado de Especialización en DevOps & Cloud impartido por la Universidad Internacional de La Rioja .

El objetivo principal del proyecto consiste en diseñar una solución completamente automatizada para el aprovisionamiento de infraestructura y el despliegue de aplicaciones sobre Microsoft Azure, aplicando principios DevOps e Infraestructura como Código .

A diferencia de un despliegue tradicional realizado manualmente, toda la infraestructura y los procesos desarrollados en este proyecto pueden reproducirse utilizando únicamente código, scripts y herramientas de automatización.

Durante el desarrollo se implementaron dos escenarios completamente independientes que permiten comparar diferentes estrategias de despliegue de aplicaciones contenerizadas:

- Despliegue sobre una Máquina Virtual Linux utilizando Podman y Ansible: <https://57.156.66.2028443/> (usr: unir/pwd: alumno2026
- Despliegue sobre Azure Kubernetes Service ) utilizando Kubernetes: <http://57.156.50.215/>

Ambos escenarios utilizan un registro privado de imágenes mediante Azure Container Registry  y comparten una infraestructura creada automáticamente mediante Terraform.

El resultado obtenido corresponde a una plataforma completamente automatizada capaz de:

- Aprovisionar infraestructura.
- Construir imágenes de contenedores.
- Publicar imágenes en Azure Container Registry.

Desplegar aplicaciones automáticamente.

- Validar el resultado del despliegue.
- Gestionar nuevas versiones mediante scripts Bash.

2. Objetivos

Objetivo general

Diseñar e implementar una plataforma DevOps completamente automatizada sobre Microsoft Azure utilizando herramientas modernas de Infraestructura como Código, automatización de configuración y orquestación de contenedores.

Objetivos específicos

- Automatizar el aprovisionamiento de infraestructura mediante Terraform.
- Implementar un registro privado de imágenes utilizando Azure Container Registry.
- Automatizar el despliegue de aplicaciones sobre una Máquina Virtual Linux utilizando Podman y Ansible.
- Implementar un clúster administrado de Kubernetes mediante Azure Kubernetes Service.
- Desarrollar una aplicación web desplegable sobre Kubernetes.
- Automatizar completamente el ciclo Build → Push → Deploy.
- Aplicar buenas prácticas de versionamiento mediante Git y GitHub.
- Validar automáticamente cada despliegue realizado.

3. Repositorio Git

Todo el código fuente desarrollado durante el proyecto se encuentra disponible en el siguiente repositorio GitHub.

Repositorio <https://github.com/guarrior-labs/cp2-devops-azure-unir>

En este repositorio se encuentran versionados:

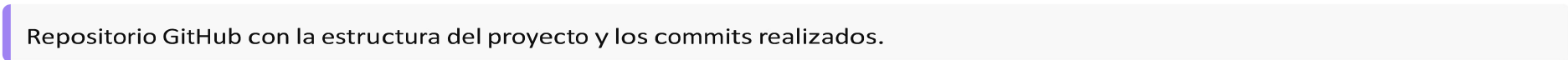
- Infraestructura Terraform.
- Configuración Ansible.
- Scripts Bash.
- Aplicación Podman.
- Aplicación Kubernetes.
- Dockerfiles.

Archivos YAML.

Documentación del proyecto.

Los commits fueron realizados siguiendo un criterio incremental, permitiendo mantener trazabilidad sobre cada funcionalidad implementada durante el desarrollo.

Captura 1 y 2



Repositorio GitHub con la estructura del proyecto y los commits realizados.

cp2-devops-azure-unir

Public

Pin

Watch 0

Fork 0

Star 0

main

1 Branch

0 Tags

Go to file

Add file

Code

guarrior-labs

- Actualiza las páginas de VM/Podman y AKS.

280832a · 1 minute ago

5 Commits

ansible	Commit Inicial - CP2 Infraestructura y Despliegue automatico...	10 hours ago
app	- Actualiza las páginas de VM/Podman y AKS.	1 minute ago
scripts	Commit Inicial - CP2 Infraestructura y Despliegue automatico...	10 hours ago
terraform	Commit Inicial - CP2 Infraestructura y Despliegue automatico...	10 hours ago
.gitignore	Feature - Juego DevOps Flask + API REST + Docker listo para...	5 hours ago
LICENSE	Commit Inicial - CP2 Infraestructura y Despliegue automatico...	10 hours ago
README.md	Feature - Juego DevOps Flask + API REST + Docker listo para...	5 hours ago

About

Caso Práctico 2 del Diplomado de Especialización en DevOps & Cloud sobre Azure

Readme

View license

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Commits

main

All users

All time

Commits on Jul 13, 2026

- Actualiza las páginas de VM/Podman y AKS.

guarrior-labs committed 4 minutes ago

280832a

Feature - Automatización completa del despliegue en AKS con validaciones y actualización automática

guarrior-labs committed 2 hours ago

e5575d0

Feature - Manifiestos Kubernetes para despliegue en AKS

guarrior-labs committed 4 hours ago

2adbf24

Feature - Juego DevOps Flask + API REST + Docker listo para AKS

guarrior-labs committed 5 hours ago

ef656ed

Commits on Jul 12, 2026

Commit Inicial - CP2 Infraestructura y Despliegue automatico en Podman

guarrior-labs committed 10 hours ago

ec9e0a4

4. Arquitectura General

La solución implementada se diseñó siguiendo una arquitectura basada en automatización completa del ciclo DevOps.

Terraform actúa como herramienta de Infraestructura como Código encargándose de crear todos los recursos necesarios sobre Microsoft Azure.

Una vez disponible la infraestructura, Azure Container Registry funciona como repositorio central de imágenes para ambos escenarios de despliegue.

Posteriormente, cada aplicación utiliza una estrategia distinta de despliegue:

- Máquina Virtual Linux utilizando Podman y Ansible.
- Azure Kubernetes Service utilizando Kubernetes.

Finalmente, ambos escenarios exponen la aplicación para su validación desde Internet.

Diagrama 1. Arquitectura general

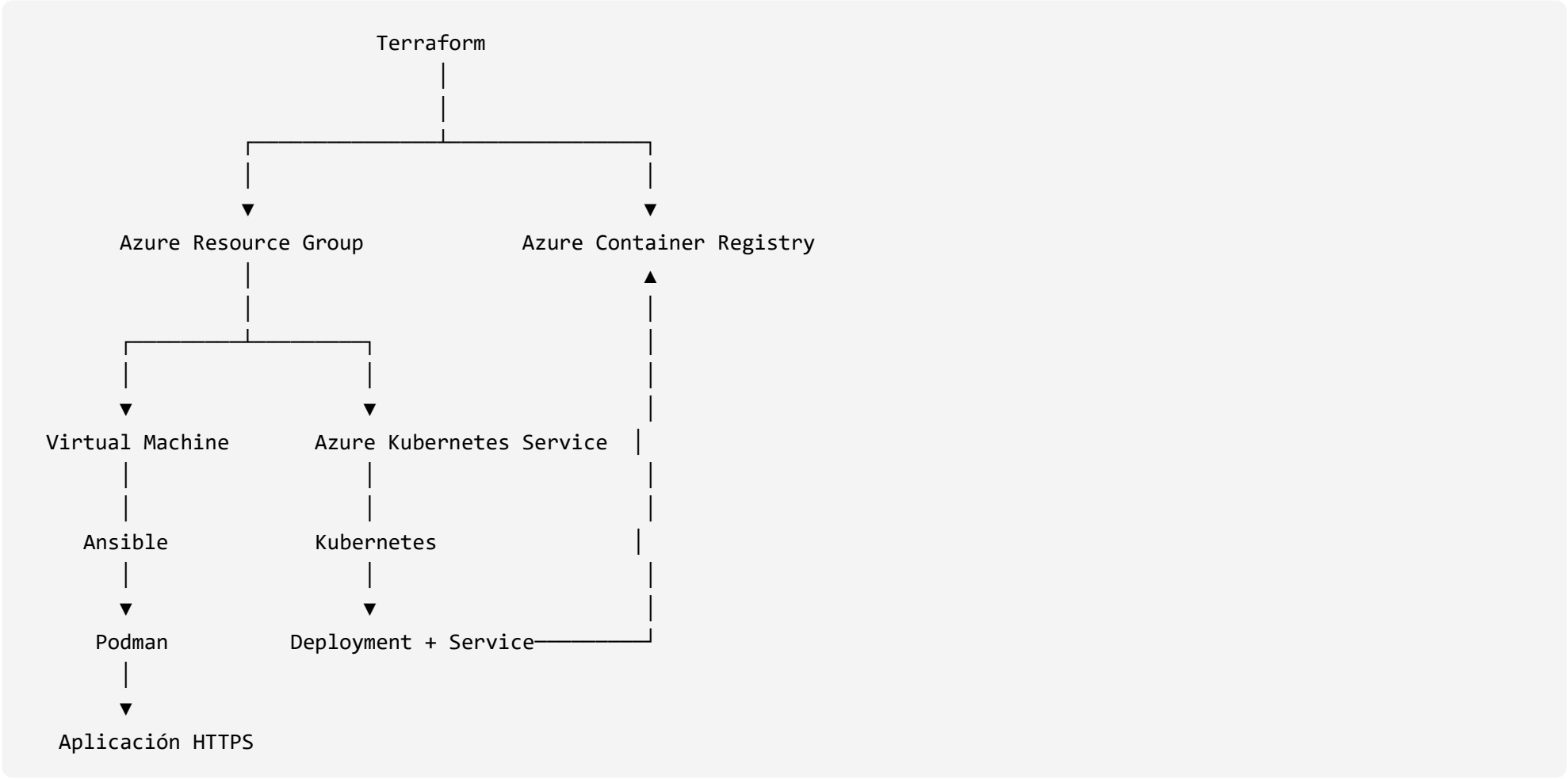
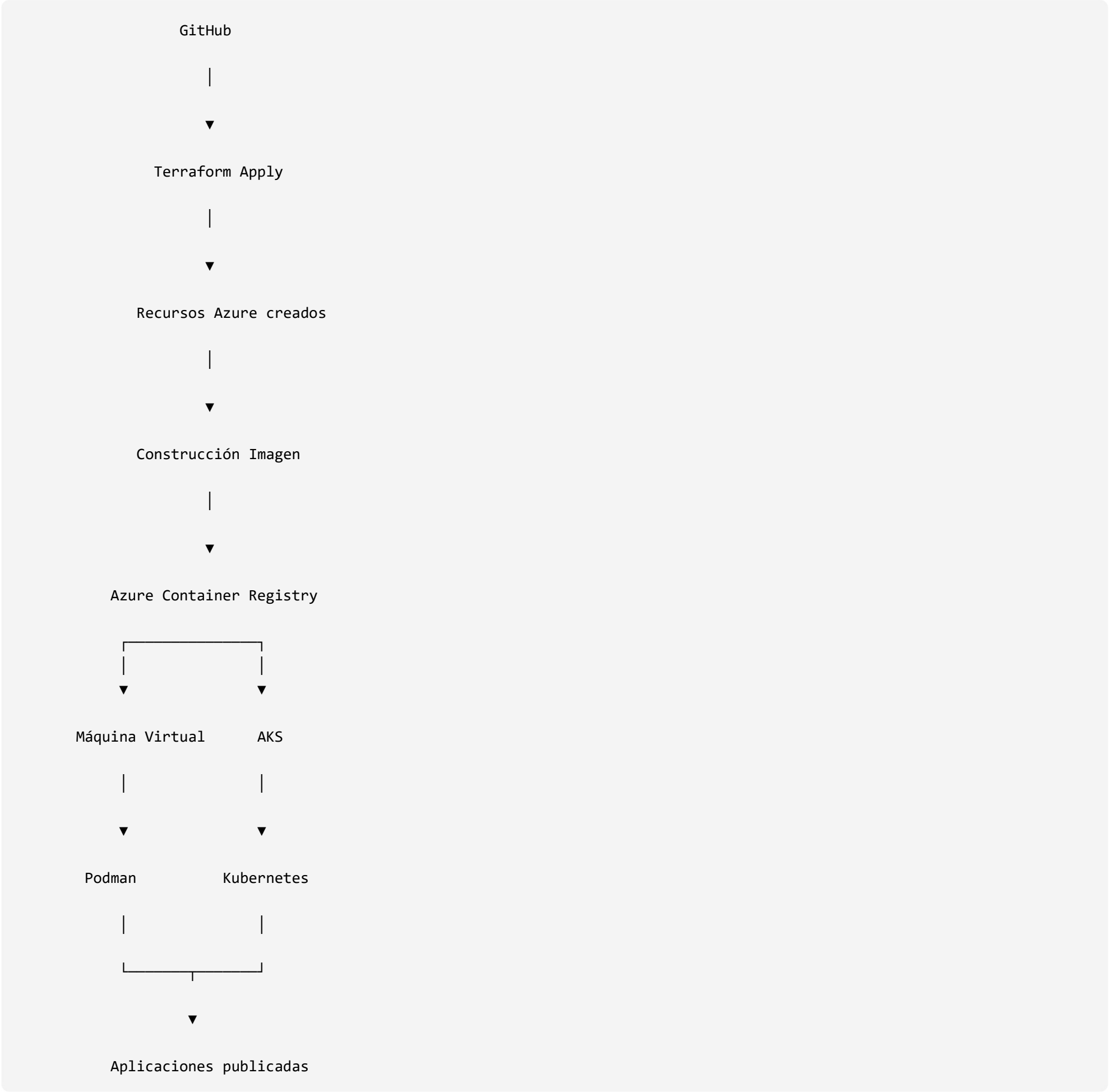


Diagrama 2. Flujo DevOps



6. Recursos desplegados en Azure

Durante la ejecución del proyecto fueron aprovisionados automáticamente los siguientes recursos.

Recurso	Nombre
Resource Group	rg-cp2-devops
Virtual Network	vnet-cp2
Network Security Group	nsg-cp2
Public IP VM	57.156.66.202
Azure Virtual Machine	vm-cp2
Azure Container Registry	cp2guarrioracr2026
Azure Kubernetes Service	aks-cp2-devops
Public IP AKS	57.156.50.215

Todos estos recursos fueron creados mediante Terraform, garantizando reproducibilidad y evitando configuraciones manuales.

La automatización permitió reconstruir completamente el entorno utilizando únicamente archivos de configuración versionados en Git.

Captura 3

Captura del Resource Group mostrando todos los recursos creados.

☐	Nombre ↑		Tipo	Grupo de recursos	Ubicación	Suscripción
☐	 aks-agentpool-74448043-nsg	...	Grupo de seguridad de red	mc_rg-cp2-devops_aks-cp2...	Chile Central	Azure for Students
☐	 aks-cp2-devops	...	Servicio de Kubernetes	rg-cp2-devops	Chile Central	Azure for Students
☐	 aks-cp2-devops-agentpool	...	Identidad administrada	MC_rg-cp2-devops_aks-cp2...	Chile Central	Azure for Students
☐	 aks-default-13172854-vmss	...	Conjunto de escalas de má...	MC_rg-cp2-devops_aks-cp2...	Chile Central	Azure for Students
☐	 aks-vnet-74448043	...	Red virtual	MC_rg-cp2-devops_aks-cp2...	Chile Central	Azure for Students
☐	 ba76bc28-4245-4b31-ba20-a797295d107c	...	Dirección IP pública	MC_rg-cp2-devops_aks-cp2...	Chile Central	Azure for Students
☐	 cp2guarrioracr2026	...	Container registry	rg-cp2-devops	Chile Central	Azure for Students
☐	 kubernetes	...	Equilibrador de carga	mc_rg-cp2-devops_aks-cp2...	Chile Central	Azure for Students
☐	 kubernetes-a0f60220d8ab64f229592e8ed27aeb87	...	Dirección IP pública	mc_rg-cp2-devops_aks-cp2...	Chile Central	Azure for Students
☐	 NetworkWatcher_chilecentral	...	Network Watcher	NetworkWatcherRG	Chile Central	Azure for Students
☐	 nic-cp2	...	Interfaz de red	rg-cp2-devops	Chile Central	Azure for Students
☐	 nsg-cp2	...	Grupo de seguridad de red	rg-cp2-devops	Chile Central	Azure for Students
☐	 osdisk-cp2	...	Disco	RG-CP2-DEVOPS	Chile Central	Azure for Students
☐	 pip-cp2	...	Dirección IP pública	rg-cp2-devops	Chile Central	Azure for Students
☐	 pvc-5fa59003-a070-40be-939d-2d3d0412c9ee	...	Disco	MC_rg-cp2-devops_aks-cp2...	Chile Central	Azure for Students
☐	 vm-cp2	...	Máquina virtual	rg-cp2-devops	Chile Central	Azure for Students
☐	 vnet-cp2	...	Red virtual	rg-cp2-devops	Chile Central	Azure for Students

Grupos de recursos

...

Resumir mis costos por servicio

Exportar grupos de recursos con Bicep o Terraform

¿Cómo administrar los cambios con las herramien

+ Crear

 Administrar vista

▼

 Actualizar

 Exportar a CSV

 Abrir consulta

|

 Asignar etiquetas

 Agregar a un grupo de servicios

 Filtrar por cualquier ca...

Suscripción es igual que **todo**

Ubicación es igual que **todo**

×

+

 Agregar filtro

☐	Nombre ↑		Suscripción	Ubicación
☐	 MC_rg-cp2-devops_aks-cp2-devops_chilecentral	...	Azure for Students	Chile Central
☐	 NetworkWatcherRG	...	Azure for Students	Chile Central
☐	 rg-cp2-devops	...	Azure for Students	Chile Central

Captura 4

Captura del Azure Container Registry.



cp2guarrioracr2026

Container registry





Comprobar la extracción de clústeres de AKS de este registro

>>

→ Mover

✖ Eliminar

^ Información esencial

Grupo de recursos (mover) : rg-cp2-devops

Ubicación : Chile Central

Suscripción (mover) : Azure for Students

Id. de suscripción : f2b6310b-4a08-40c9-94bf-16f6b6670afd

Eliminación temporal (ve...) : Deshabilitado

Etiquetas (editar) : environment : casopractico2

Servidor de inicio de sesi... : cp2guarrioracr2026.azurecr.io

Fecha de creación : 11/7/2026, 9:45 GMT-5

Estado de aprovisionami... : Succeeded

Plan de precios : Basic

Ámbito de etiqueta de n... : No seguro

Captura 5

Captura del clúster Azure Kubernetes Service.

☐	Nombre ↑	Tipo	Suscripción	Grupo de recu...	Ubicación	Administrador ...	Estado de pote...	Espacios de no...	Versión de Ku...	
☐	 aks-cp2-devops	...	Servicio de Kub...	Azure for Stude...	rg-cp2-devops	Chile Central	 Agregar a f...	 En ejecución	0	1.35.5

^ Essentials

Grupo de recursos : rg-cp2-devops

Estado de energía : En ejecución

Estado de la operación ... : Se realizó correctamente

Suscripción : Azure for Students

Ubicación : Chile Central

Id. de suscripción : f2b6310b-4a08-40c9-94bf-16f6b6670afd

Administrador de flota : Haga clic aquí para asignar

Etiquetas (editar) : environment : casopractico2

Versión de Kubernetes : 1.35.5

Dirección del servidor d... : akscp2-uchqqb5b.hcp.chilecentral.azmk8s.io

SKU : Base

Plan de tarifa : Gratis

Configuración de red : Superposición de Azure CNI

Grupos de nodos : 1 grupo de nodos

Registros de contenedor : cp2guarrioracr2026

Hora de creación : 13 de julio de 2026 a las 00:47

Captura 6

Captura de la Máquina Virtual creada por Terraform.

☐	Nombre	Tipo ↑	Grupo de recursos	Ubicación	Suscripción	
☐	 vm-cp2	...	Máquina virtual	rg-cp2-devops	Chile Central	Azure for Students

7. Descripción del proceso de despliegue

La solución implementada contempla dos escenarios de despliegue completamente independientes. Ambos utilizan la misma infraestructura base creada mediante Terraform y un registro privado de imágenes administrado por Azure Container Registry [ACR]. Sin embargo, cada uno emplea tecnologías diferentes para demostrar distintos enfoques de automatización.

7.1 Escenario 1 – Despliegue sobre Máquina Virtual utilizando Podman

El primer escenario consiste en desplegar una aplicación web sobre una Máquina Virtual Linux creada automáticamente mediante Terraform.

Una vez creada la infraestructura, la configuración del servidor se automatiza mediante Ansible, mientras que la aplicación se ejecuta dentro de un contenedor administrado por Podman.

El proceso completo se encuentra automatizado mediante el script:

```
scripts/deploy-podman.sh
```

Este script permite ejecutar todo el proceso utilizando un único comando:

```
./scripts/deploy-podman.sh v19
```

SHELL

donde el parámetro recibido corresponde a la versión de la imagen que será publicada y desplegada.

Flujo del despliegue

El script realiza automáticamente las siguientes actividades:

- 1. Obtiene la configuración del proyecto desde Terraform.
- 2. Construye la imagen Podman.
- 3. Inserta automáticamente la versión de la aplicación.
- 4. Etiqueta la imagen.
- 5. Realiza autenticación segura contra Azure Container Registry.
- 6. Publica la imagen.
- 7. Actualiza automáticamente la configuración utilizada por Ansible.
- 8. Ejecuta el Playbook.
- 9. Verifica el contenedor desplegado.
- 10. Comprueba la versión instalada.
- 11. Valida el servicio HTTPS.
- 12. Verifica el contenido publicado.

Todo este procedimiento se realiza sin intervención manual.

Automatización implementada

El script incorpora múltiples validaciones para garantizar la confiabilidad del proceso.

Entre ellas se encuentran:

- Verificación de comandos instalados.
- Obtención automática de parámetros desde Terraform.
- Construcción automática de la imagen.
- Inserción dinámica del número de versión.
- Validación del Push hacia Azure Container Registry.
- Validación del contenedor desplegado.

Validación HTTP y HTTPS.

Verificación de la versión publicada.

En caso de producirse cualquier error durante el proceso, el despliegue finaliza inmediatamente mostrando un mensaje descriptivo.

Aplicación desplegada

La aplicación desplegada sobre la Máquina Virtual corresponde a un servidor Nginx configurado con:

- HTTPS.
- Certificado SSL.
- Autenticación Básica.
- Página HTML con versión automática.

Versionamiento dinámico durante cada despliegue.

Esto permitió demostrar la automatización completa del ciclo Build → Push → Deploy utilizando Podman.

Captura 7

Captura de la aplicación ejecutándose mediante HTTPS sobre la Máquina Virtual.



Insertar captura del script `deploy-podman.sh` finalizando correctamente.

```
Validando imagen desplegada...
Imagen en ejecución: cp2guarrioracr2026.azurecr.io/casopractico2:v18
Versión esperada : v18

[6/6]Esperando que la aplicación esté disponible...

=====
DEPLOY FINALIZADO CORRECTAMENTE
=====
Imagen      : cp2guarrioracr2026.azurecr.io/casopractico2:v18
Versión     : v18
Servidor    : 57.156.66.202
HTTPS       : OK
URL HTTP    : http://57.156.66.202:8080
URL HTTPS   : https://57.156.66.202:8443
=====
```

7.2 Escenario 2 – Despliegue sobre Azure Kubernetes Service (AKS)

Como segundo escenario se implementó una aplicación completamente diferente orientada a demostrar las capacidades de Kubernetes.

El despliegue se realiza sobre un clúster administrado de Azure Kubernetes Service [AKS].

La aplicación fue desarrollada utilizando:

- Python
 - Flask
 - HTML
 - CSS
- JavaScript

y se ejecuta dentro de un Deployment de Kubernetes.

Toda la automatización se encuentra implementada mediante el script:

```
scripts/deploy-aks.sh
```

El despliegue completo puede ejecutarse mediante:

```
./scripts/deploy-aks.sh v10
```

SHELL

Flujo del despliegue

El script automatiza completamente las siguientes tareas:

- [?] Lectura automática de los outputs de Terraform.
- [?] Construcción de la imagen.
- [?] Inserción automática del número de versión.
- [?] Publicación en Azure Container Registry.
- [?] Validación del Push.
- [?] Obtención automática de credenciales del clúster AKS.

?? Creación o actualización automática del Deployment.

?? Actualización de la imagen mediante Rolling Update.

?? Espera automática del Rollout.

??? Verificación del Pod.

??? Verificación del PersistentVolumeClaim.

??? Verificación del Service.

??? Validación HTTP.

??? Validación de la API REST.

??? Resumen automático del despliegue.

Al igual que en el escenario anterior, todo el proceso puede ejecutarse mediante un único comando.

Kubernetes

Para este escenario fueron implementados los siguientes recursos:

Deployment

Encargado de mantener la aplicación ejecutándose y administrar los Rolling Updates.

Service

Permite publicar la aplicación hacia Internet utilizando un LoadBalancer administrado por Azure.

PersistentVolumeClaim

Se implementó un volumen persistente para almacenar el Hall of Fame de la aplicación.

Gracias a este componente la información permanece disponible incluso después de reiniciar el Pod.

Rolling Update

Una de las características más importantes implementadas fue la actualización automática de imágenes mediante:

```
kubectl set image deployment/cp2-game ...
```

SHELL

Esto permitió desplegar nuevas versiones sin necesidad de eliminar el Deployment existente.

Durante las pruebas se validaron correctamente múltiples versiones consecutivas (v1, v2, v3...v8), observándose el cambio automático de versión desde la interfaz web.

Captura del clúster AKS funcionando.



aks-default-13172854-vmss

Conjunto de escalas de máquina virtual



Recomendar un tamaño para este conjunto de escalad

>>

 Actualizar

 Inicio

 Reiniciar

 Detener

 Eliminar

 Recrear imagen

 Mover



 [Advisor \(1 de 1\): Enable application health monitoring for Virtual Machine Scale Sets \(VMSS\)](#)

^ Essentials

Grupo de recursos [\(mover\)](#)

[MC_rg-cp2-devops aks-cp2-devops chilecentral](#)

Estado

[1 de 1 correctas](#)

Suscripción

[Azure for Students](#)

Ubicación [\(mover\)](#)

Chile Central

Id. de suscripción

f2b6310b-4a08-40c9-94bf-16f6b6670afd

Sistema operativo

Linux

Cambiar el tamaño

[Standard D2s v3 \(1 instancia\)](#)

Dirección IP pública

[57.156.48.34](#)

Dirección IP pública (IPv6)

-

Red virtual/subred

[aks-vnet-74448043/aks-subnet](#)

Modo de orquestación

Uniform

Hora de creación

13/7/2026, 5:50:48 UTC



Perfil de máquina virtual

Sistema operativo	Linux
Grupo de reserva de capacidad	-
Hibernación	Deshabilitado



Azure Spot

Azure Spot	Deshabilitado
------------	---------------



Disponibilidad y escalado

Zona de disponibilidad	- (editar)
Zona extendida	-
Grupo con ubicación por proximidad	-
Estado de coubicación	-
Grupo de hosts	-
Recuento de instancias	1
Escalado	Manual
Directiva de reducción horizontal	Valor predeterminado
Aprovisionamiento excesivo	-
Recuento de dominios de error	1
Único grupo de selección de ubicación	-



Estado

Estado de aprovisionamiento	Correcto
Estado de energía	1de 1 instancias en ejecución



Redes

IP pública	57.156.48.34
Dirección IP pública (IPv6)	-
Red virtual/subred	aks-vnet-74448043/aks-subnet



Cambiar el tamaño

Cambiar el tamaño	Standard_D2s_v3
vCPU	2
RAM	8 GiB
Subprocesos por núcleo	2



Detalles de la imagen de origen

Publicador de la imagen de origen	-
Oferta de la imagen de origen	-
Plan de imagen de origen	-

Captura 10

Captura de los Pods ejecutándose (`kubectl get pods`).

```
guarrior@DESKTOP-GFCIC5L:~/workspace/cp2$ kubectl get pods -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
cp2-game-6bc77cf8d9-6s6z5 1/1 Running 0 64m 10.244.0.50 aks-default-13172854-vmss000000 <none> <none>
```

Captura 11

Captura del Service (`kubectl get svc`).

```
guarrior@DESKTOP-GFCIC5L:~/workspace/cp2$ kubectl get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
cp2-game-service LoadBalancer 10.0.79.94 57.156.50.215 80:32041/TCP 4h30m
kubernetes ClusterIP 10.0.0.1 <none> 443/TCP 5h7m
```


Captura del PersistentVolumeClaim (`kubectl get pvc`).

```
guarrior@DESKTOP-GFCIC5L:~/workspace/cp2$ kubectl get svc
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
cp2-game-service                   LoadBalancer        10.0.79.94      57.156.50.215    80:32041/TCP     4h31m
kubernetes                         ClusterIP            10.0.0.1        <none>           443/TCP          5h9m
```

8. Descripción de la aplicación desarrollada

A diferencia de la aplicación desplegada sobre la Máquina Virtual, la solución implementada para Kubernetes fue diseñada específicamente para demostrar conceptos DevOps de una forma interactiva.

La aplicación recibe el nombre de:

DevOps Deployment Challenge

El objetivo consiste en simular la ejecución de un Pipeline DevOps mediante un pequeño juego.

El usuario debe reconstruir correctamente el orden de las etapas del pipeline para conseguir un despliegue exitoso.

Funcionamiento

Al iniciar la aplicación el usuario debe registrar un nombre.

Cada nombre únicamente puede registrarse una vez.

Posteriormente aparecen cuatro actividades del Pipeline DevOps en un orden aleatorio.

El jugador debe seleccionar cada etapa en el orden correcto.

Cuando las cuatro etapas han sido seleccionadas, se habilita el botón Ejecutar Deploy.

La aplicación valida automáticamente el orden elegido.

Si la secuencia es correcta:

- se muestra un despliegue exitoso; se
- incrementa el contador del jugador; se
- actualiza automáticamente el Hall of Fame.

En caso contrario, la aplicación informa cuál era el orden correcto.

Hall of Fame

El Hall of Fame almacena el número total de despliegues exitosos realizados por cada participante.

La información permanece disponible gracias al PersistentVolumeClaim implementado en Kubernetes.

Esto permitió demostrar el funcionamiento del almacenamiento persistente dentro del clúster.

API REST

La aplicación incorpora una pequeña API REST utilizada por el frontend.

Los principales endpoints implementados fueron:

Método	Endpoint	Función
GET	/api/ranking	Consulta el Hall of Fame
POST	/api/player	Registra nuevos jugadores
POST	/api/win	Incrementa el número de despliegues exitosos

Durante el proceso de despliegue, el script **deploy-aks.sh** valida automáticamente que la API responda correctamente antes de finalizar la ejecución.

Versionado automático

Cada imagen generada incorpora automáticamente el número de versión correspondiente al despliegue.

De esta forma fue posible validar visualmente que cada Rolling Update realmente publicaba una nueva versión de la aplicación.

Captura 13

Captura de la pantalla principal del juego.



Captura 14

Insertar captura del Hall of Fame.

Hall of Fame

 **DanielG** - 3 deploy(s)

 **Laura** - 0 deploy(s)

 **Juan** - 0 deploy(s)

 **Gaby** - 2 deploy(s)

Captura 15

Captura mostrando la versión desplegada (v1, v2, v3...).

No es seguro

57.156.50.215



DESPLIEGUE AUTOMATIZADO - VERSIÓN: v8

No es seguro

57.156.50.215

Información del despliegue

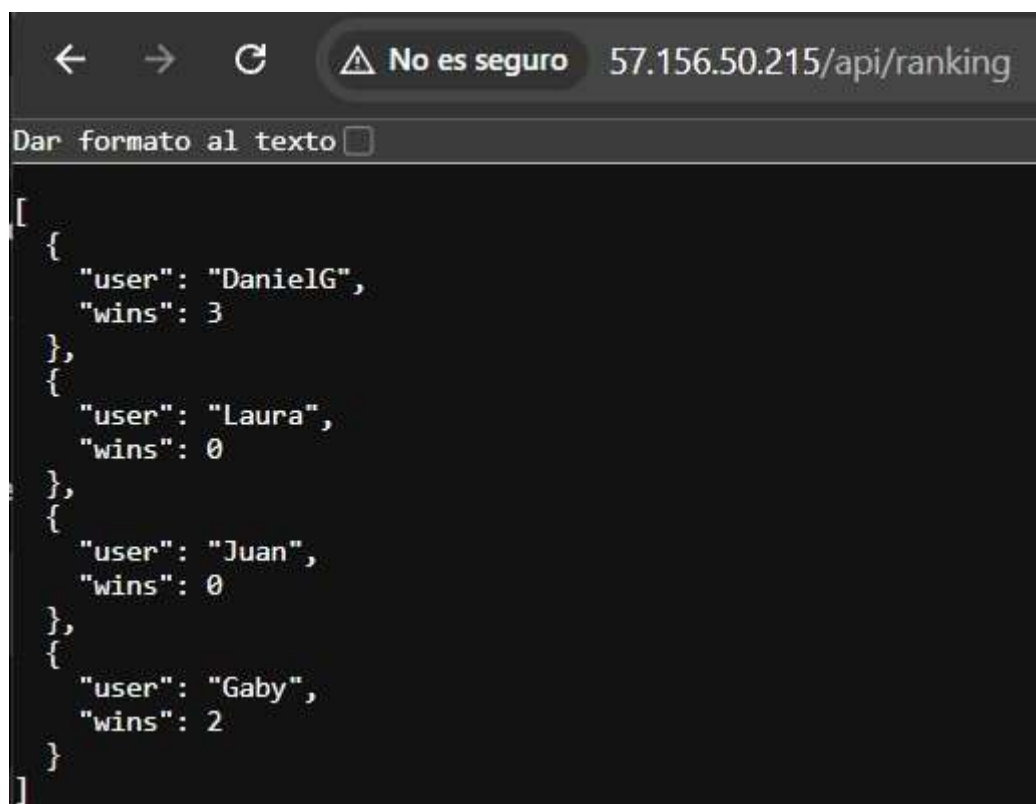
- **Versión:** v8
- **Autor:** Daniel Guerrero
- **Proyecto:** Caso Práctico 2 - DevOps & Cloud
- **Universidad:** UNIR
- **Año:** 2026

Nombre del participante

DanielG

Iniciar reto

Captura de la API REST respondiendo correctamente.



```
[
  {
    "user": "DanielG",
    "wins": 3
  },
  {
    "user": "Laura",
    "wins": 0
  },
  {
    "user": "Juan",
    "wins": 0
  },
  {
    "user": "Gaby",
    "wins": 2
  }
]
```

9. Automatización implementada

Uno de los principales objetivos del proyecto consistió en minimizar la intervención manual durante el ciclo completo de despliegue.

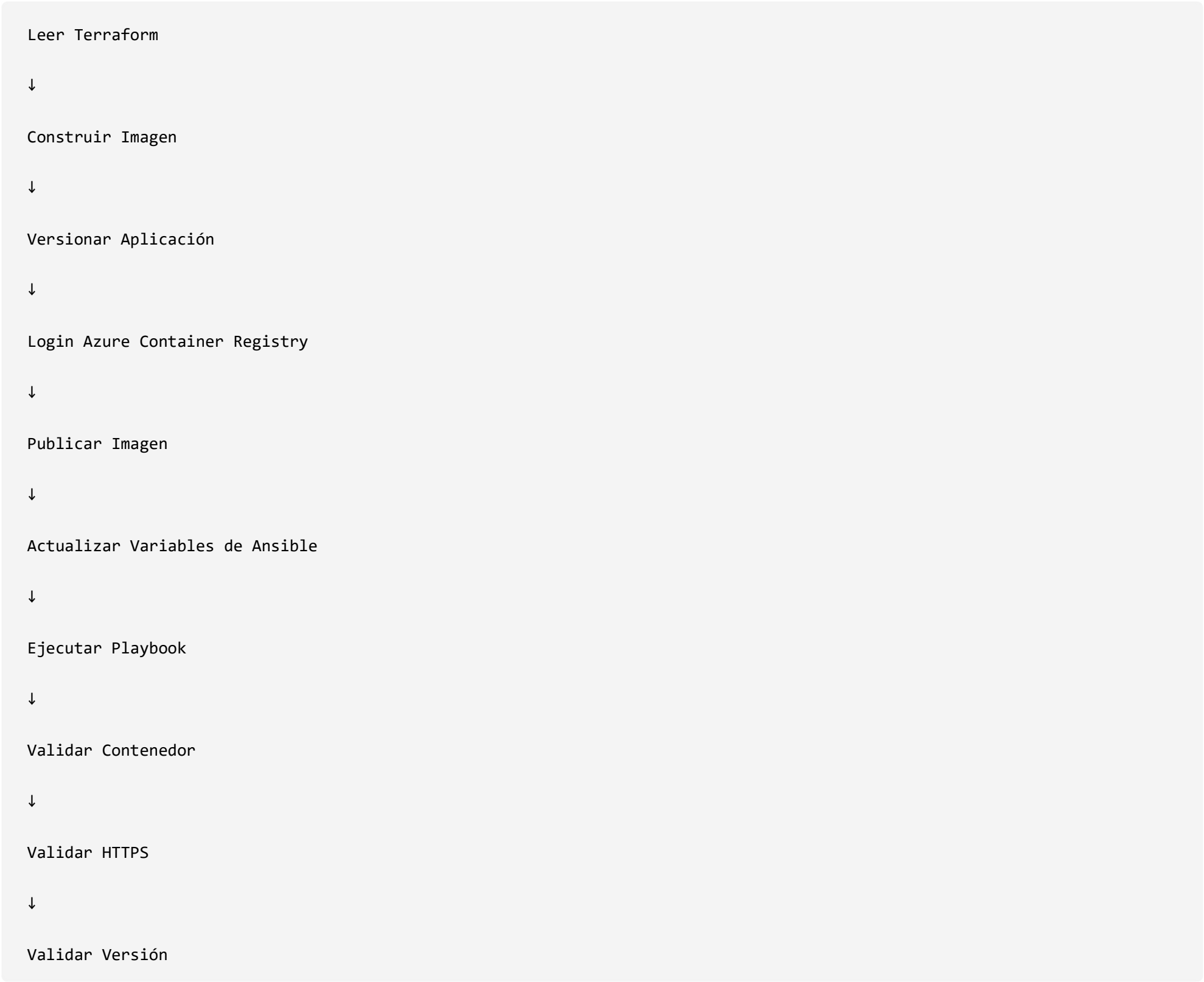
Para lograrlo se desarrollaron dos scripts Bash independientes que automatizan completamente ambos escenarios implementados.

El resultado obtenido fue un proceso repetible, versionado y confiable, alineado con los principios DevOps estudiados durante el diplomado.

9.1 Script deploy-podman.sh

El script **deploy-podman.sh** automatiza completamente el despliegue de la aplicación sobre la Máquina Virtual Linux.

Su funcionamiento puede resumirse en el siguiente flujo:



Durante su ejecución el script realiza automáticamente las siguientes actividades:

- Descubre la ubicación del proyecto.
- Lee la IP pública de la VM desde Terraform.
- Obtiene automáticamente las credenciales del Azure Container Registry.
- Construye una nueva imagen Podman.
- Inserta dinámicamente el número de versión.
- Publica la imagen en ACR.
- Actualiza la imagen utilizada por Ansible.
- Ejecuta el Playbook.
- Comprueba la imagen desplegada.
- Valida HTTPS.
- Comprueba que la versión publicada corresponde con la esperada.
- Finaliza mostrando un resumen del despliegue.

Gracias a este proceso fue posible reducir completamente la configuración manual del despliegue.

Captura 17

Insertar captura del script `deploy-podman.sh` ejecutándose correctamente.


```
Validando imagen desplegada...
Imagen en ejecución: cp2guarrioracr2026.azurecr.io/casopractico2:v18
Versión esperada : v18

[6/6]Esperando que la aplicación esté disponible...

=====
DEPLOY FINALIZADO CORRECTAMENTE
=====
Imagen      : cp2guarrioracr2026.azurecr.io/casopractico2:v18
Versión     : v18
Servidor    : 57.156.66.202
HTTPS       : OK
URL HTTP    : http://57.156.66.202:8080
URL HTTPS   : https://57.156.66.202:8443
=====
```

9.2 Script deploy-aks.sh

Para el despliegue sobre Kubernetes se desarrolló un segundo script completamente independiente.

Este script automatiza el ciclo completo Build → Push → Deploy sobre Azure Kubernetes Service.

Su funcionamiento puede resumirse mediante el siguiente flujo:

```
Terraform
↓
Construcción Imagen
↓
Versionado
↓
Push Azure Container Registry
↓
Conexión AKS
↓
Deployment
↓
Rolling Update
↓
Validaciones
↓
Aplicación publicada
```

Durante su ejecución realiza automáticamente:

- Obtención de parámetros desde Terraform.
- Construcción de la imagen.
- Inserción automática del número de versión.
- Publicación en Azure Container Registry.
- Verificación de la imagen publicada.

Obtención de credenciales del clúster AKS.

Creación del Deployment durante el primer despliegue.

- Actualización automática del Deployment en despliegues posteriores.
- Rolling Update.
- Verificación de Pods.
- Verificación del Service.
- Verificación del PersistentVolumeClaim.
- Validación HTTP.

Validación de la API REST.

Validación del contenido publicado.

Presentación de un resumen completo del despliegue.

La automatización permitió ejecutar múltiples despliegues consecutivos únicamente modificando el número de versión recibido como parámetro.

Captura 18

Captura del script `deploy-aks.sh` finalizando correctamente.

```
=====
DEPLOY FINALIZADO CORRECTAMENTE
=====

AKS           : aks-cp2-devops
Resource Group : rg-cp2-devops
ACR           : cp2guarrioracr2026.azurecr.io

Deployment    : cp2-game
Service       : cp2-game-service
PVC           : hall-of-fame-pvc

Namespace     : default
Imagen        : cp2guarrioracr2026.azurecr.io/casopractico2-game:v8
Versión       : v8
IP Pública    : 57.156.50.215

Pod           : cp2-game-6bc77cf8d9-6s6z5
Estado        : Running
HTTP          : 200
API REST      : OK

URL
-----
http://57.156.50.215
API REST: http://57.156.50.215/api/ranking

Hall of Fame persistente : SI

=====

Tiempo total : 59 segundos
```

10. Hallazgos y soluciones implementadas

Durante el desarrollo del proyecto surgieron diferentes inconvenientes técnicos que requirieron análisis, investigación y ajustes en la solución implementada.

Cada uno de estos hallazgos permitió fortalecer el proyecto y mejorar significativamente la calidad de la automatización obtenida.

Hallazgo 1

Restricción de regiones en Azure

Problema

La suscripción Azure for Students no permitía crear recursos en determinadas regiones debido a políticas institucionales.

Causa

Azure Policy restringía las regiones autorizadas para el despliegue. Solución

Se identificó la política aplicada mediante Azure CLI y se modificó la variable de ubicación utilizada por Terraform hacia la región Chile Central.

Resultado

Terraform logró desplegar correctamente toda la infraestructura.

Hallazgo 2

Outputs de Terraform no disponibles

Problema

Los scripts no obtenían automáticamente la IP pública ni las credenciales del ACR. Causa

Los nuevos outputs agregados al proyecto aún no existían dentro del archivo de estado (`terraform.tfstate`).

Solución Se ejecutó:

```
terraform apply -refresh-only
```

SHELL

para sincronizar el estado sin modificar la infraestructura existente.

Resultado

Los scripts comenzaron a obtener automáticamente toda la configuración requerida.

Hallazgo 3

Login inseguro hacia Azure Container Registry

Problema

Inicialmente la contraseña era enviada mediante el parámetro `-p`.

Causa

Este mecanismo expone la contraseña dentro del historial de comandos.

Solución

Se reemplazó por:

```
podman login --password-stdin
```

SHELL

Resultado

Se mejoró la seguridad del proceso de autenticación.

Hallazgo 4

Rutas absolutas dentro de los scripts

Problema

Los scripts dependían del directorio desde donde eran ejecutados.

Causa

Las rutas estaban codificadas manualmente.

Solución

Se implementó el descubrimiento automático del directorio utilizando:

```
BASH_SOURCE
```

SHELL

Resultado

Los scripts pueden ejecutarse desde cualquier ubicación.

Hallazgo 5

Errores durante la validación HTTP

Problema

El proceso de validación fallaba aun cuando la aplicación respondía correctamente.

Causa

El archivo Bash contenía caracteres invisibles (tabulaciones) que interrumpían la continuidad del comando `curl`.

Solución

Se normalizó completamente el archivo eliminando dichos caracteres.

Resultado

Las validaciones comenzaron a funcionar correctamente.

Hallazgo 6

Versionado manual de la aplicación

Problema

Era necesario editar manualmente el número de versión mostrado por la aplicación.

Causa

La versión estaba escrita directamente dentro del HTML.

Solución

Se implementó un archivo plantilla (`index.template.html`) junto con reemplazo automático mediante `sed`.

Resultado

Cada despliegue actualiza automáticamente la versión mostrada.

Hallazgo 7

Actualización manual del Deployment de Kubernetes

Problema

Cada nueva versión obligaba a modificar manualmente el manifiesto YAML.

Causa

El Deployment tenía una referencia fija hacia la imagen.

Solución Se implementó:

```
kubect1 set image
```

SHELL

para actualizar automáticamente el Deployment.

Resultado

Se habilitaron Rolling Updates completamente automáticos.

Hallazgo 8

Persistencia del Hall of Fame

Problema

El Hall of Fame desaparecía al reiniciar el Pod.

Causa

Los datos eran almacenados únicamente dentro del sistema de archivos del contenedor.

Solución

Se implementó un PersistentVolumeClaim.

Resultado

La información permanece disponible incluso después de recrear los Pods.

Hallazgo 9

Usuarios duplicados

Problema

Era posible registrar múltiples jugadores con el mismo nombre.

Causa

No existía validación dentro de la API REST.

Solución

Se implementó una búsqueda previa antes de registrar un nuevo usuario.

Resultado

Cada usuario únicamente puede registrarse una vez.

Hallazgo 10

Automatización incompleta del despliegue sobre AKS

Problema

Inicialmente el despliegue requería ejecutar múltiples comandos manuales. Causa
Las operaciones sobre Kubernetes no estaban integradas en un único flujo.

Solución

Se desarrolló el script `deploy-aks.sh`.

Resultado

Actualmente el despliegue completo puede ejecutarse mediante un único comando.

Lecciones aprendidas

Durante el desarrollo del proyecto se confirmó que la automatización representa uno de los pilares fundamentales de DevOps.

Cada problema encontrado evidenció la importancia de construir procesos repetibles y verificables, donde las validaciones automáticas permiten detectar errores antes de afectar el entorno productivo.

También se comprobó que Kubernetes ofrece un nivel superior de flexibilidad respecto a un despliegue tradicional sobre una Máquina Virtual, especialmente gracias a características como los Rolling Updates, los Deployments y el almacenamiento persistente.

Finalmente, el uso conjunto de Terraform, Azure, Podman, Ansible y Kubernetes permitió integrar diferentes tecnologías dentro de un único flujo automatizado, acercándose a un escenario muy similar al encontrado en proyectos reales de ingeniería de software.

Captura 19

Captura del Hall of Fame persistente después de actualizar la aplicación.

=====

DEPLOY FINALIZADO CORRECTAMENTE

=====

AKS : aks-cp2-devops

Resource Group : rg-cp2-devops

ACR : cp2guarrioracr2026.azurecr.io

Deployment : cp2-game

Service : cp2-game-service

PVC : hall-of-fame-pvc

Namespace : default

Imagen : cp2guarrioracr2026.azurecr.io/casopractico2-game:v9

Versión : v9

IP Pública : 57.156.50.215

Pod : cp2-game-9dbd44784-wbwdn

Estado : Running

HTTP : 200

API REST : OK

URL

http://57.156.50.215

API REST: http://57.156.50.215/api/ranking

Hall of Fame persistente : SI

=====

Tiempo total : 57 segundos

guarrior@DESKTOP-GFCIC5L:~/workspace/cp2\$

DESPLIEGUE AUTOMATIZADO - VERSIÓN: v9

DevOps Deployment Challenge

Tu misión ha comenzado... ¿Estás listo para el reto? Demuestra tus conocimientos de DevOps seleccionando el orden correcto de los pasos del despliegue en Azure Kubernetes Service (AKS). Analiza cada opción, toma la mejor decisión y completa la misión con éxito. Identifica la secuencia perfecta del despliegue, evita errores en el pipeline y completa el desafío DevOps.

Información del despliegue

Versión: v9

Autor: Daniel Guerrero

Proyecto: Caso Práctico 2 - DevOps & Cloud

Universidad: UNIR

Auto: 2026

Nombre del participante

Escribe tu nombre

Iniciar

Pasos disponibles

Aquí aparecerán los pasos.

Orden seleccionado

Ningún paso seleccionado.

Seleccionar Deploy

Resultado

Esperando...

Hall of Fame

DanielD - 3 deploy(s)

Laure - 0 deploy(s)

Juan - 0 deploy(s)

Osby - 2 deploy(s)

Captura 20

Captura del Rolling Update mostrando la actualización de versión sin interrupciones.

```
Namespace      : default
Imagen        : cp2guarrioracr2026.azurecr.io/casopractico2-game:v9
Versión       : v9
IP Pública    : 57.156.50.215

Pod           : cp2-game-9dbd44784-wbwdn
Estado        : Running
HTTP          : 200
API REST      : OK

URL
-----
http://57.156.50.215
API REST: http://57.156.50.215/api/ranking

Hall of Fame persistente : SI
=====

Tiempo total : 57 segundos
guarrior@DESKTOP-GFCIC5L:~/workspace/cp2$ kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
cp2-game-9dbd44784-wbwdn           1/1     Running   0           7m9s

^Cguarrior@DESKTOP-GFCIC5L:~/workspace/cp2$ kubectl rollout history deployment/cp2-game
deployment.apps/cp2-game
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
3          <none>
4          <none>
5          <none>
6          <none>
7          <none>
8          <none>

guarrior@DESKTOP-GFCIC5L:~/workspace/cp2$ kubectl rollout status deployment/cp2-game
deployment "cp2-game" successfully rolled out
guarrior@DESKTOP-GFCIC5L:~/workspace/cp2$
```

11. Resultados obtenidos

El desarrollo del Caso Práctico 2 permitió cumplir satisfactoriamente los objetivos planteados al inicio del proyecto, integrando múltiples tecnologías dentro de una arquitectura completamente automatizada.

A lo largo del desarrollo se implementaron procesos de Infraestructura como Código , automatización de configuración, construcción de imágenes, despliegue de aplicaciones contenerizadas y orquestación mediante Kubernetes.

El resultado obtenido demuestra que es posible desplegar una misma infraestructura utilizando diferentes estrategias tecnológicas manteniendo un alto nivel de automatización y reduciendo significativamente la intervención manual.

Resultados alcanzados

Durante el proyecto se logró implementar exitosamente:

Infraestructura

- Creación automática del Resource Group.
 - Creación de la red virtual.
 - Configuración del Network Security Group.
 - Aprovisionamiento automático de la Máquina Virtual Linux.
 - Creación automática de Azure Container Registry.
 - Creación automática del clúster Azure Kubernetes Service.
-

Automatización

- Automatización del aprovisionamiento mediante Terraform.
 - Automatización del despliegue sobre Podman.
 - Automatización del despliegue sobre Kubernetes.
 - Automatización del versionado de aplicaciones.
 - Automatización del Push hacia Azure Container Registry.
 - Automatización del Rolling Update.
 - Automatización de las validaciones posteriores al despliegue.
-

Contenedores

Se desarrollaron dos aplicaciones diferentes.

Escenario 1

Servidor Nginx protegido mediante:

- HTTPS
 - Certificado SSL
 - Autenticación Básica
 - Versionado automático
-

Escenario 2

Aplicación Flask denominada:

DevOps Deployment Challenge

Incluye:

- Registro de jugadores.
 - Validación de usuarios.
 - Hall of Fame.
 - API REST.
 - Persistencia mediante PersistentVolumeClaim.
 - Rolling Updates automáticos.
-

Control de versiones

El proyecto fue administrado completamente mediante Git.

Se utilizó GitHub como repositorio remoto para mantener el historial de cambios, permitiendo la trazabilidad de todas las funcionalidades implementadas.

Evidencias de funcionamiento

Durante las pruebas finales se verificó correctamente:

- Creación automática de infraestructura.
- Construcción de imágenes.
- Publicación en Azure Container Registry.
- Despliegue sobre Máquina Virtual.
- Despliegue sobre Azure Kubernetes Service.
- Funcionamiento del servidor HTTPS.
- Funcionamiento de la API REST.
- Persistencia del Hall of Fame.

Actualización automática de versiones.

Rolling Updates.

Validaciones automáticas posteriores al despliegue.

Todos estos resultados fueron comprobados mediante pruebas funcionales y verificaciones automáticas incorporadas dentro de los scripts desarrollados.

12. Licencia

El presente proyecto utiliza la licencia MIT License.

La licencia MIT es una licencia de software libre ampliamente utilizada debido a su simplicidad y flexibilidad.

Permite:

- Uso personal.
- Uso académico.
- Uso comercial.
- Copia del software.
- Modificación.

Distribución.

Publicación.

La única condición consiste en conservar el aviso de copyright y el texto de la licencia original en cualquier redistribución del software.

La licencia no ofrece garantías sobre el funcionamiento del software y exime al autor de cualquier responsabilidad derivada de su utilización.

13. Conclusiones

Terraform demostró ser una herramienta sólida para automatizar completamente el aprovisionamiento de infraestructura, eliminando configuraciones manuales y garantizando la reproducibilidad del entorno.

La implementación de Azure Container Registry permitió centralizar la gestión de imágenes utilizadas por ambos escenarios de despliegue, facilitando el versionamiento y la distribución de las aplicaciones.

El escenario basado en Podman permitió comprender el proceso tradicional de despliegue sobre una Máquina Virtual, mientras que Azure Kubernetes Service evidenció las ventajas de la orquestación moderna de contenedores mediante Deployments, Services y PersistentVolumeClaims.

Uno de los aspectos más relevantes del proyecto fue el desarrollo de dos scripts completamente automatizados (`deploy-podman.sh` y `deployaks.sh`), capaces de ejecutar el ciclo completo de construcción, publicación, despliegue y validación utilizando un único comando.

La aplicación DevOps Deployment Challenge permitió demostrar de manera práctica conceptos fundamentales como la persistencia de datos, el consumo de una API REST, el versionamiento automático y los Rolling Updates sobre Kubernetes.

Desde una perspectiva personal, este proyecto representó el primer acercamiento práctico a tecnologías como Terraform, Podman, Ansible y Kubernetes. El proceso implicó enfrentar diversos desafíos técnicos cuya resolución fortaleció significativamente la comprensión de los principios DevOps y de las buenas prácticas de automatización.

Más allá de cumplir con los objetivos académicos propuestos, el proyecto dejó una reflexión importante: la automatización no consiste únicamente en reducir tareas repetitivas, sino en construir procesos confiables, reproducibles y mantenibles. Ese cambio de enfoque constituye uno de los aprendizajes más valiosos obtenidos durante el desarrollo del Caso Práctico 2.

14. Referencias

Ansible Community. 2025. *Ansible Documentation*. <https://docs.ansible.com/>

HashiCorp. 2025. *Terraform Documentation*. <https://developer.hashicorp.com/terraform/docs>

Kubernetes Authors. 2025. *Kubernetes Documentation*. <https://kubernetes.io/docs/>

Microsoft. 2025. *Azure Container Registry Documentation*. <https://learn.microsoft.com/azure/container-registry/>

Microsoft. 2025. *Azure Kubernetes Service Documentation*. <https://learn.microsoft.com/azure/aks/>

Microsoft. 2025. *Azure Virtual Machines Documentation*. <https://learn.microsoft.com/azure/virtual-machines/>

Podman Team. 2025. *Podman Documentation*. <https://podman.io/>

Python Software Foundation. 2025. *Flask Documentation*. <https://flask.palletsprojects.com/> Git

Project. 2025. *Git Documentation*. <https://git-scm.com/docs>

Anexos

Anexo A. Estructura final del proyecto

```
cp2/
├── app/
│   ├── podman/
│   └── kubernetes/
├── ansible/
├── docs/
├── scripts/
│   ├── deploy-podman.sh
│   └── deploy-aks.sh
├── terraform/
├── README.md
├── LICENSE
└── .gitignore
```

Anexo B. Scripts desarrollados

Durante el proyecto se desarrollaron los siguientes componentes de automatización:

- `deploy-podman.sh`
 - `deploy-aks.sh`
 - Playbooks de Ansible.
 - Archivos Terraform.
 - Manifiestos Kubernetes.
 - Dockerfiles para ambos escenarios.
-
-

Declaración final

Declaro que el presente trabajo corresponde a un desarrollo original realizado para el Caso Práctico 2 del Diplomado de Especialización en DevOps & Cloud de la Universidad Internacional de La Rioja .

Las decisiones técnicas, la implementación, las pruebas realizadas y la documentación presentada reflejan el proceso de aprendizaje y la integración de los conocimientos adquiridos durante el diplomado.

Fin del documento